Primitives and references

Prof. Guilherme Cavalcanti

Agenda

- what you can declare as a variable;
- what you can put in a variable;
- what you can do with a variable;
- what happens on Java’s heap.

Variables **store** data/values for a later use in your program.

Typical uses of variables

- as object state (instance variables/attributes)
- as local variables (variables declared within a method)

Typical uses of variables

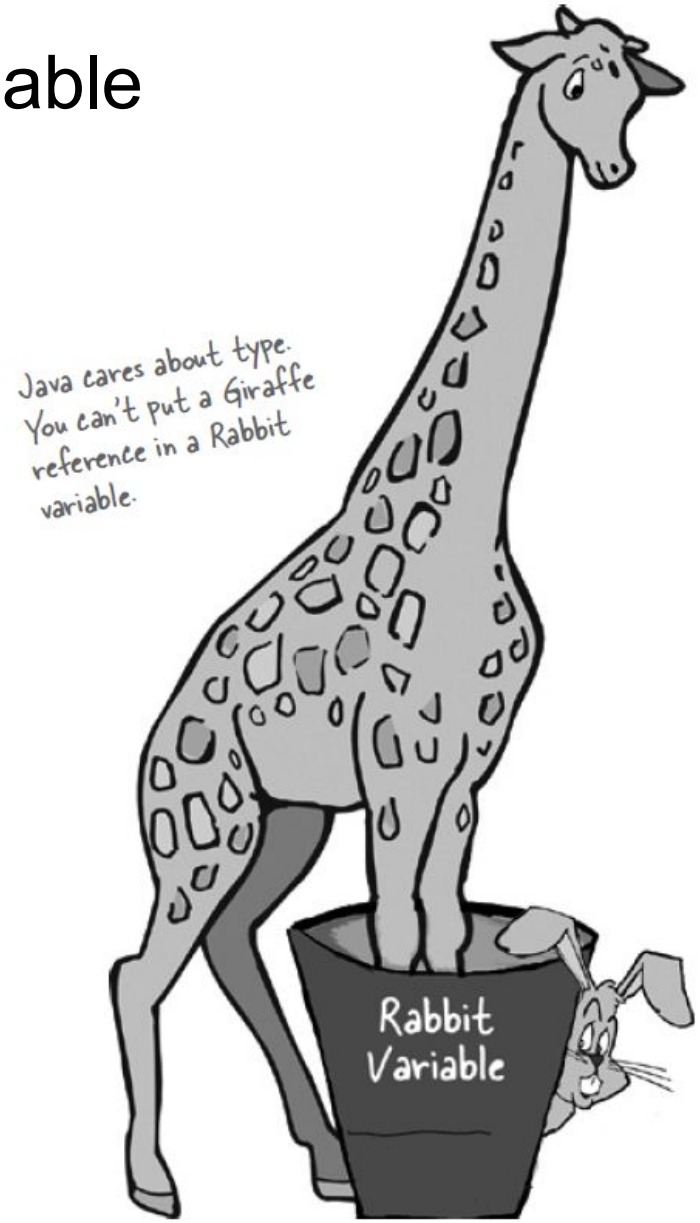
- as object state (instance variables)
- as local variables (variables declared within a method)
- as arguments (values sent to a method by the calling code)
- as return types (values sent back to the caller of the method)

Values of variables

- as simple primitive values (example, type int)
- as complex values (like a String or an array)

What if you have a PetOwner object with a Dog instance variable? Or a Car with an Engine?

Declaring a variable



The compiler can spot most problems:

```
Rabbit hopper = new Giraffe();
```

Don’t expect that to compile. Thankfully.

Two types of Java Variables

- **Primitives** hold fundamental values including integers, booleans, and floating-point numbers.

- **Object references** hold references to objects.

variables must have a type

variables must have a name

```
int count;
```

type

name

Note: when you see a statement like: “an object of type X,” think of *type* and *class* as synonyms.

A variable is just a ***container***. It ***holds*** something.

Primitive Types

Type	Bit Depth	Value Range
boolean and char		
boolean	(JVM-specific)	***true*** or ***false***
char	16 bits	0 to 65535
numeric (all are signed)		
integer		
byte	8 bits	-128 to 127
short	16 bits	-32768 to 32767
int	32 bits	-2147483648 to 2147483647
long	64 bits	-huge to huge
floating point		
float	32 bits	varies
double	64 bits	varies

Primitive declarations with assignments:

```
int x;
x = 234;
byte b = 89;
boolean isFun = true;
double d = 3456.98;
char c = 'f';
int z = x;
boolean isPunkRock;
isPunkRock = false;
boolean powerOn;
powerOn = isFun;
long big = 3456789L;
float f = 32.5f;
```

Note the 'f' and 'L'. With some number types, you have to specifically tell the compiler what you mean, or it might get confused between similar-looking number types. You can use upper or lowercase.

Primitive assignment.
Be sure the value can fit into the variable.

```
int x = 24;
byte b = x;
//won't work!!
```

Primitive assignment.
Be sure the value can fit into the variable.

```
int x = 24;
byte b = x;
//won't work!!
```

```
byte x = 24;
int b = x;
//does work!!
```

You can assign a value to a variable in one of several ways, including

- type a literal value after the equals sign (`x = 12`, `isGood = true`, etc.)
- assign the value of one variable to another (`x = y`)
- use an expression combining the two (`x = y + 43`)

Examples:

`int size = 32;`	declare an int named *size*, assign it the value *32*
`char initial = 'j';`	declare a char named *initial*, assign it the value *'j'*
`double d = 456.709;`	declare a double named *d*, assign it the value *456.709*
`boolean isLearning;`	declare a boolean named *isCrazy* (no assignment)
`isLearning = true;`	assign the value *true* to the previously declared *isCrazy*
`int y = x + 456;`	declare an int named *y*, assign it the value that is the sum of whatever *x* is now plus *456*

Exercise: if these lines were in a single method, which are correct?

```
1. int x = 34.5;

2. boolean boo = x;

3. int g = 17;

4. int y = g;

5. y = y + 10;

6. short s;

7. s = y;

8. byte b = 3;

9. byte v = b;

10. short n = 12;

11. v = n;

12. byte k = 128;
```

Rules for *naming*. Back away from *keywords*!

- It must start with a letter, underscore (_), or dollar sign ($). You can’t start a name with a number.
- After the first character, you can use numbers as well. Just don’t start it with a number.
- It can be anything you like, subject to those two rules, just so long as it isn’t one of Java’s reserved words.

Do not use *keywords* as name!

_	catch	double	float	int	private	super	true
abstract	char	else	for	interface	protected	switch	try
assert	class	enum	goto	long	public	synchronized	void
boolean	const	extends	if	native	return	this	volatile
break	continue	false	implements	new	short	throw	while
byte	default	final	import	null	static	throws	
case	do	finally	instanceof	package	strictfp	transient	

Java's keywords, reserved words, and special identifiers. If you use these for names, the compiler will *probably* be very, *very* upset.

Object references

Object references

- There is actually no such thing as an object variable.
- There is only an object reference variable.
- An object reference variable holds bits that represent away to access an object.
- It does not hold the object itself, but it holds something like a pointer. Or an address.

```
Dog d = new Dog();
d.bark();
```

think of this

like this

Think of a Dog reference variable as a Dog remote control. You use it to get the object to do something (invoke methods).

The 3 steps of object declaration, creation and assignment

1 Declare a reference variable

```
Dog myDog = new Dog();
```

Dog

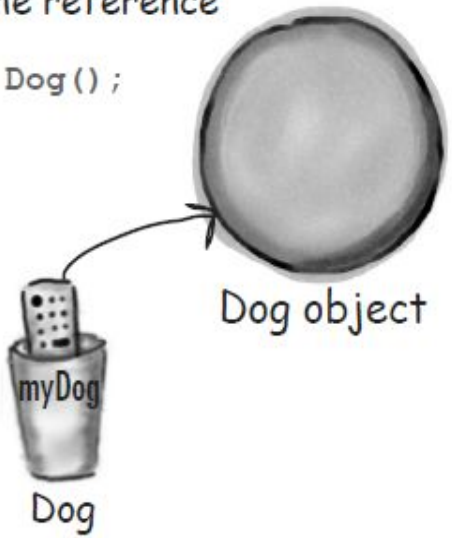
2 Create an object

```
Dog myDog = new Dog();
```

Dog object

3 Link the object and the reference

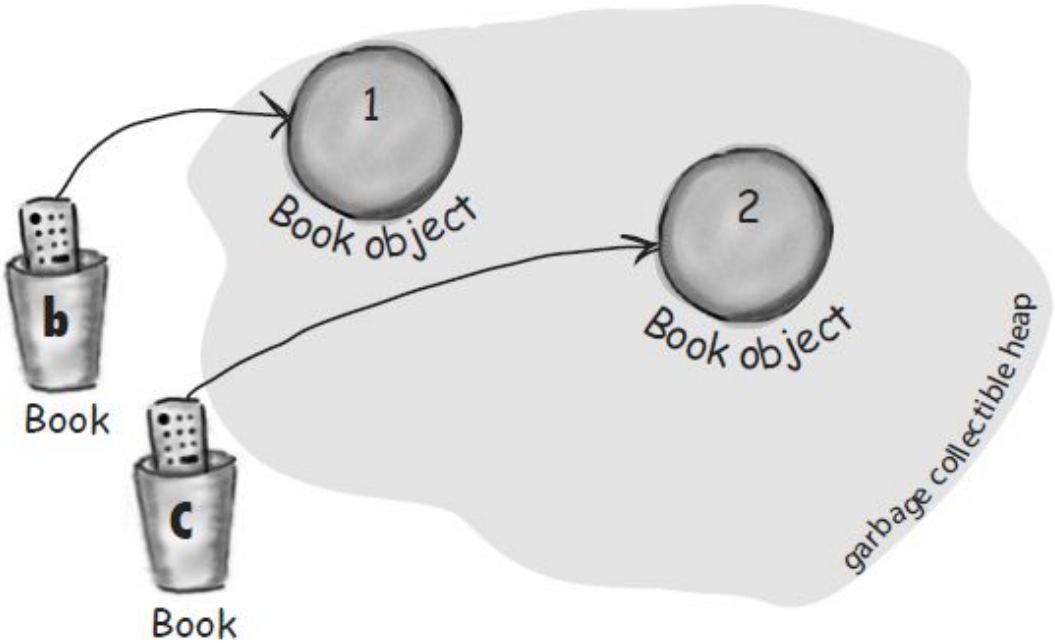
```
Dog myDog = new Dog();
```



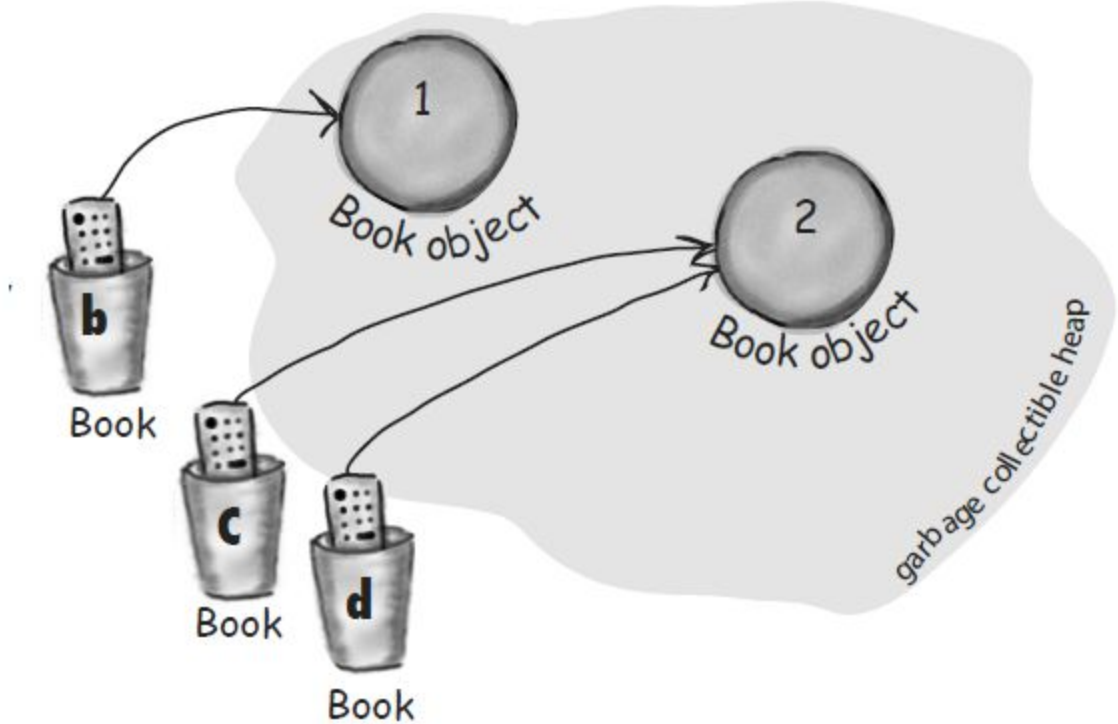
Life on the garbage-collectible heap

```
Book b = new Book();
Book c = new Book();
```

References: 2
Objects: 2



```
Book d = c;
```

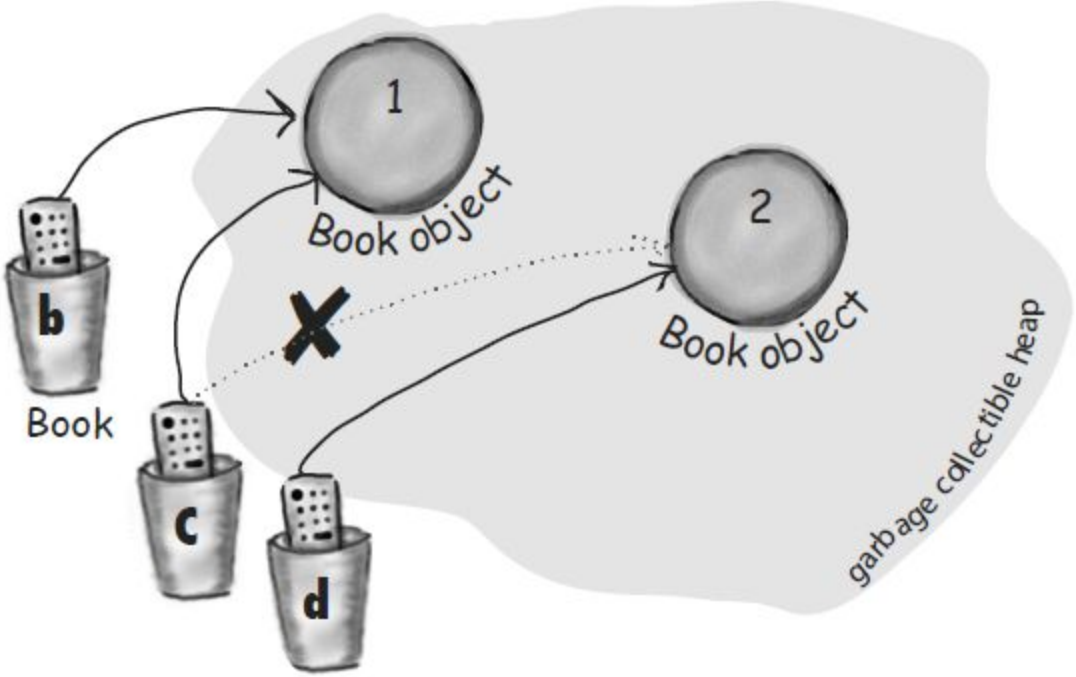


Both **c** and **d** refer to the same object.

The **c** and **d** variables hold two different copies of the same value. “Two remotes programmed to one TV.”

References: 3
Objects: 2

```
c = b;
```



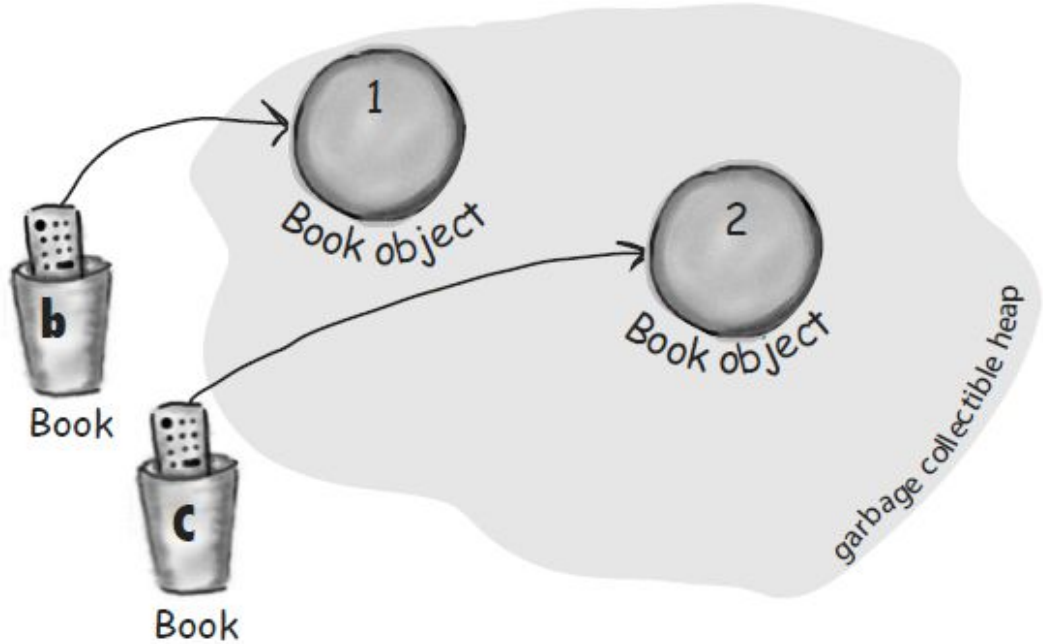
Both **b** and **c** refer to the same object.

The **c** variable no longer refers to its old Book object.

References: 3
Objects: 2

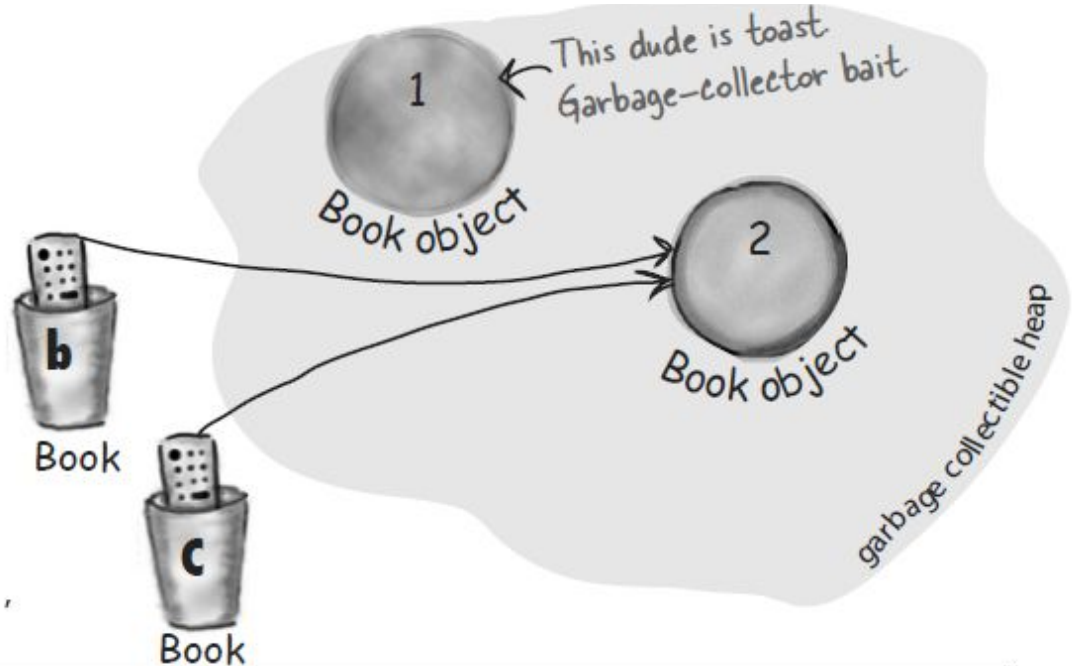
Life and death on the heap

```
Book b = new Book();
Book c = new Book();
```



Active References: 2
Reachable Objects: 2

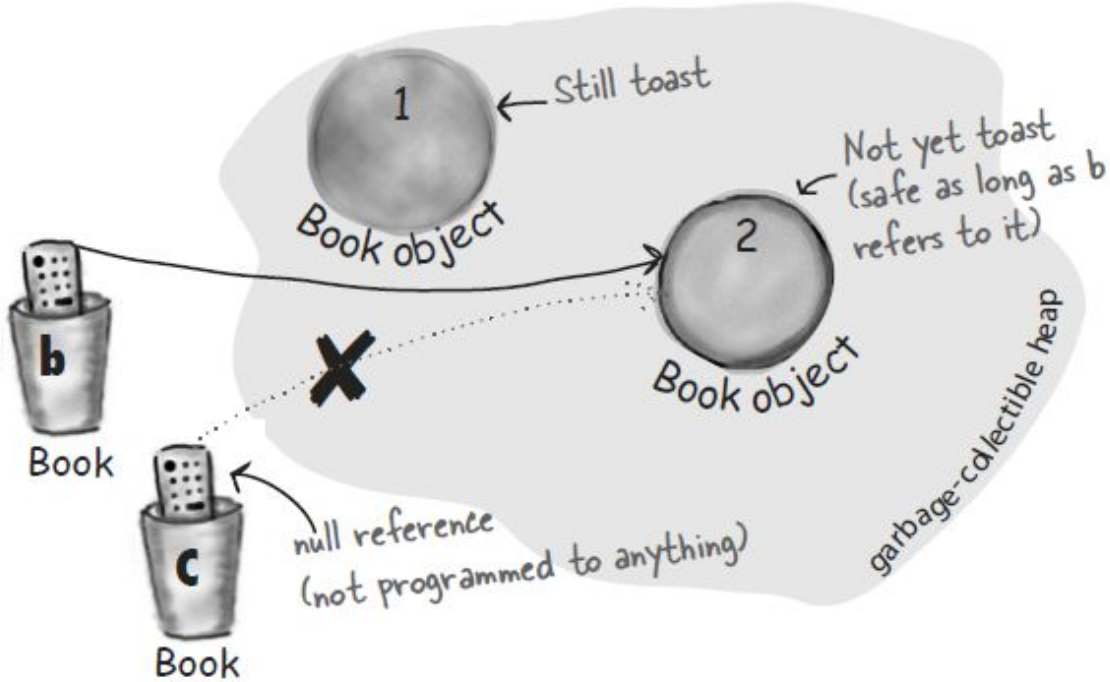
```
b = c;
```



Both **b** and **c** refer to the same object. Object 1 is abandoned and eligible for Garbage Collection (GC).

Active References: 2
Reachable Objects: 1
Abandoned Objects: 1

```
c = null;
```



Object 2 still has an active reference (**b**), and as long as it does, the object is not eligible for GC.

Active References: 1
null References: 1
Reachable Objects: 1
Abandoned Objects: 1

Arrays in Java:
a quick and efficient collection of things.

Working with *arrays*

1 Declare an int array variable. An array variable is a remote control to an array object.

```
int[] nums;
```

2 Create a new int array with a length of 7, and assign it to the previously declared `int[]` variable `nums`

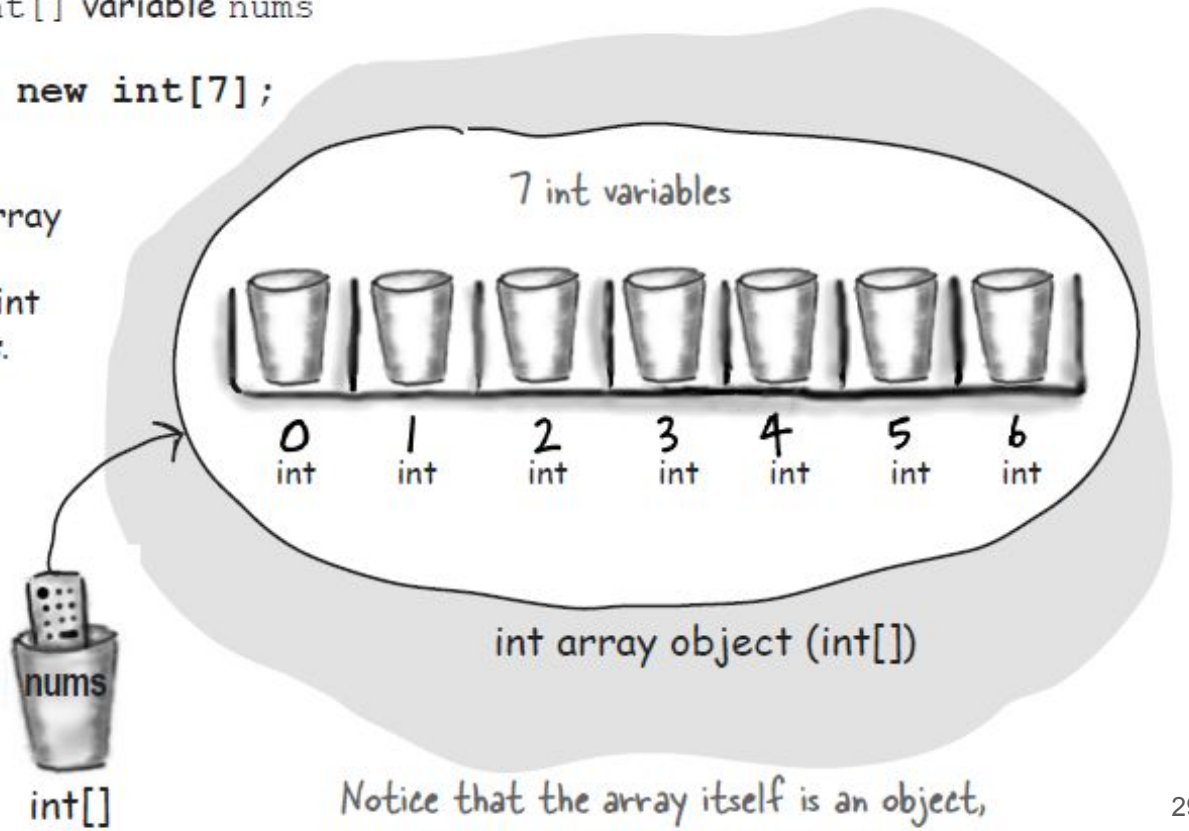
```
nums = new int[7];
```

3 Give each element in the array some int value.
Remember, elements in an int *array* are just int *variables*.

7 int variables

```
nums[0] = 6;
nums[1] = 19;
nums[2] = 44;
nums[3] = 42;
nums[4] = 10;
nums[5] = 20;
nums[6] = 1;
```

Arrays are always objects, whether they're declared to hold primitives or object references.



Notice that the array itself is an object, even though the 7 elements are primitives.

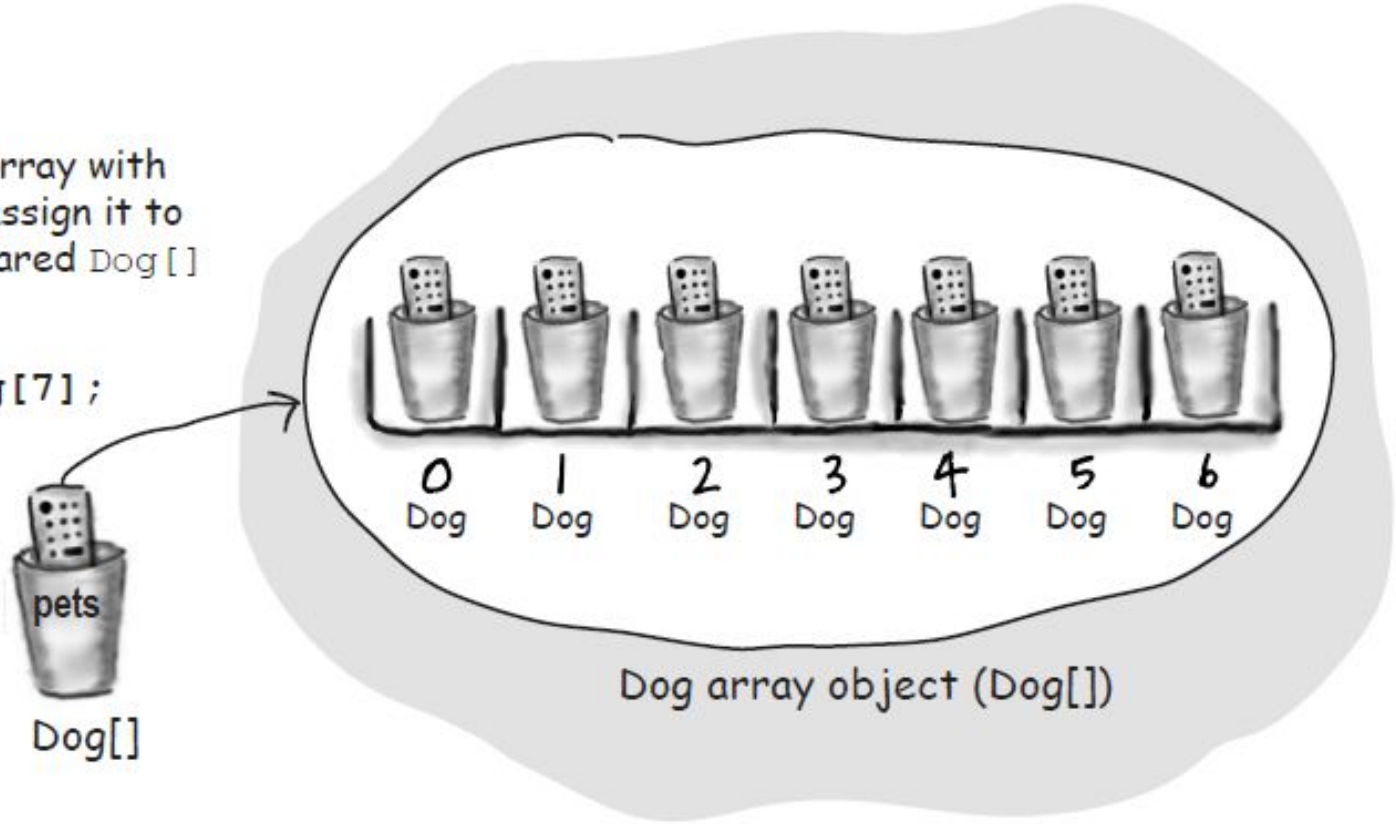
An array of objects

1. Declare a Dog array variable

```
Dog[] pets;
```

2. Create a new Dog array with a length of 7, and assign it to the previously declared `Dog[]` variable `pets`

```
pets = new Dog[7];
```

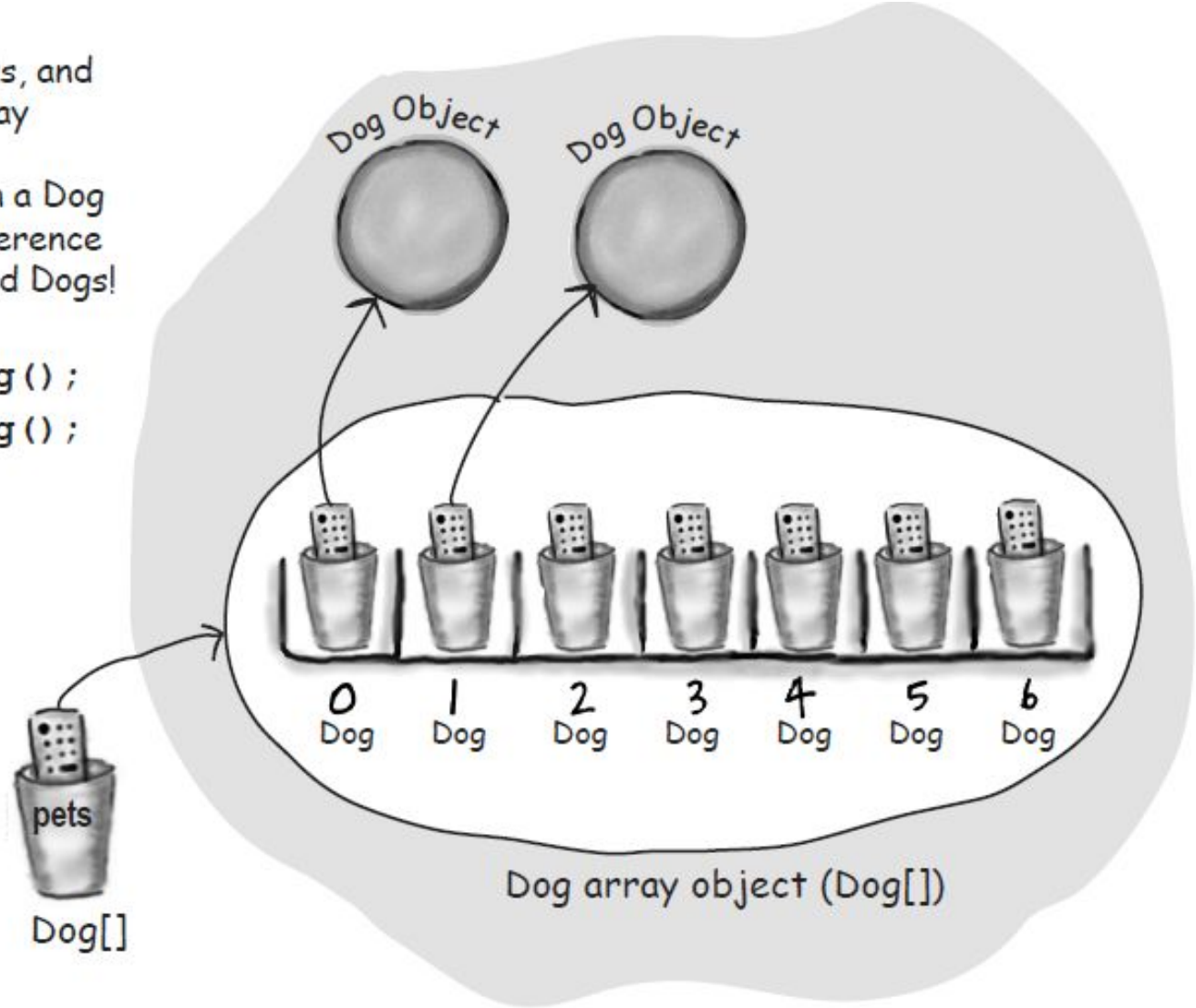


What's missing?

Dogs! We have an array of Dog *references*, but no actual Dog *objects*!

3 Create new Dog objects, and assign them to the array elements.
Remember, elements in a Dog *array* are just Dog reference *variables*. We still need Dogs!

```
pets[0] = new Dog();
pets[1] = new Dog();
```



- What is the current value of pets[2]?
- What code would make pets[3] refer to one of the two existing Dog objects?

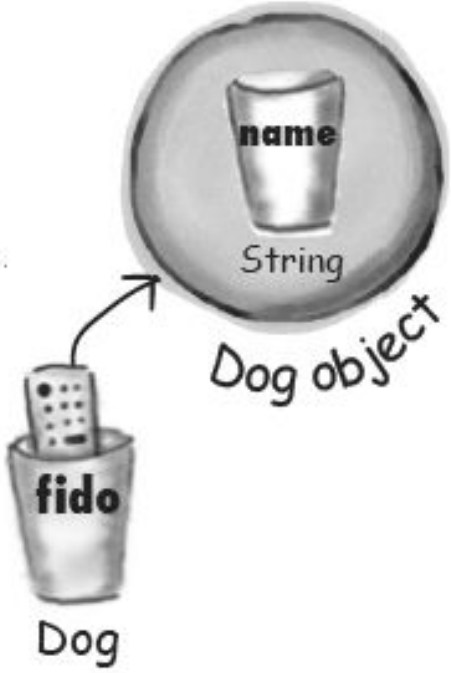
Dog
name
bark() eat() chaseCat()

Control your Dog

(with a reference variable)

```
Dog fido = new Dog();
fido.name = "Fido";

fido.bark();
fido.chaseCat();
```



What happens if the Dog is in a Dog array?

```
Dog[] myDogs = new Dog[3];
myDogs[0] = new Dog();
myDogs[0].name = "Fido";
myDogs[0].bark();
```

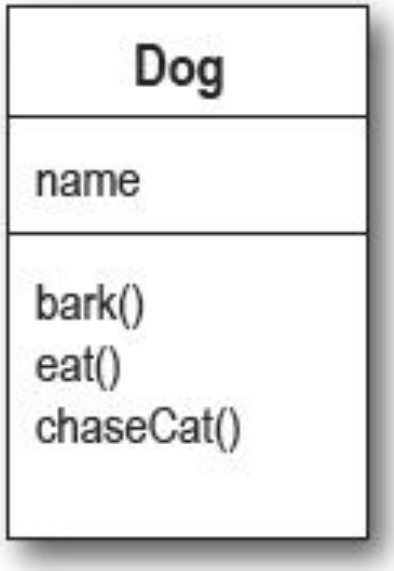
Full dog example

```
class Dog {
    String name;

    public static void main(String[] args) {
        // make a Dog object and access it
        Dog dog1 = new Dog();
        dog1.bark();
        dog1.name = "Bart";

        // now make a Dog array
        Dog[] myDogs = new Dog[3];

        // and put some dogs in it
        myDogs[0] = new Dog();
        myDogs[1] = new Dog();
        myDogs[2] = dog1;
```



```
    // now access the Dogs using the array
    // references
    myDogs[0].name = "Fred";
    myDogs[1].name = "Marge";

    // Hmmmm... what is myDogs[2] name?
    System.out.print("last dog's name is ");
    System.out.println(myDogs[2].name);

    // now loop through the array
    // and tell all dogs to bark
    int x = 0;

    while (x < myDogs.length) {
        myDogs[x].bark();
        x = x + 1;
    }
}
```

```
    public void bark() {
        System.out.println(name + " says Ruff!");
    }

    public void eat() {
    }

    public void chaseCat() {
    }
}
```

Output

```
File  Edit  Window  Help  Howl
%java Dog
null says Ruff!
last dog's name is Bart
Fred says Ruff!
Marge says Ruff!
Bart says Ruff!
```

Summary

- Variables come in two flavors: primitive and reference.

- Variables must always be declared with a name and a type.

- A primitive variable value is the bits representing the value (5, ‘a’, true, 3.1416, etc.).

 A reference variable value is the bits representing a way to get to an object on the heap.

 A reference variable is like a remote control. Using the dot operator (.) on a reference variable is like pressing a button on the remote control to access a method or instance variable.

- A reference variable has a value of null when it is not referencing any object.

- An array is always an object, even if the array is declared to hold primitives. There is no such thing as a primitive array, only an array that holds primitives.